UCLouvain

LINFO1361 - Intelligence artificielle

Projet 2 - Recherche adversariale (Oxono)

Groupe n°114
Röben Friedrich Nihat 99942500
Sogliuzzo Loris 26482510

ÉCOLE POLYTECHNIQUE DE LOUVAIN

Table des matières

1 Introduction **3**

2 Implémentation **3**
2.1 Minimax 3
2.2 Monte-Carlo Tree Search (MCTS) 4
2.3 Heuristique 4
2.4 AlphaZero 4
2.5 Athénan 5

3 Résultats et discussions **6**
3.1 Évolution de Minimax 6
3.2 Évolution de MCTS 6
3.3 Évolution de l'heuristique 6
3.4 Tournoi suisse 7

4 Conclusion **7**

Liste des tableaux

1 Architecture du réseau de neurones *R2-network* implémenté. 5
2 Résultats détaillés de l'évolution de Minimax vs MCTS basique (20 parties par match). 6
3 Résultats détaillés de l'évolution de MCTS vs Minimax basique (20 parties par match). 6
4 Résultats détaillés de l'évolution de l'heuristique avec MCTS avancé vs Minimax basique (20 parties par match). 7
5 Résultats détaillés du tournoi suisse (4 rondes, 20 parties par match). 7

1 Introduction

Durant ce projet, nous avons exploré plusieurs pistes pour concevoir une intelligence artificielle capable de jouer à Oxono, avec pour objectif clair de trouver l'agent le plus performant possible. Pour cela, nous avons implémenté plusieurs agents basés sur des paradigmes d'intelligence artificielle différents, allant des algorithmes de recherche classiques guidés par des heuristiques expertes (Minimax et Monte-Carlo Tree Search), jusqu'aux approches modernes basées sur des réseaux de neurones et l'apprentissage par renforcement (AlphaZero et Athénan). Ce rapport présente en détail les différentes implémentations que nous avons réalisées en section 2, ainsi que les résultats obtenus lors de l'évaluation de ces agents, présentés en section 3.

2 Implémentation

2.1 Minimax

Un algorithme Minimax (Russell and Norvig, 2021) basique représente le jeu et les coups possibles sous forme d'arbre. Il suppose qu'un joueur veut maximiser son gain, et minimiser celui de l'adversaire. L'algorithme choisit dès lors le coup procurant le meilleur résultat possible face à un adversaire qui joue parfaitement.

Néanmoins, la grille comporte 36 cases, sachant qu'il faut respecter la contrainte des 5 minutes par partie, il n'est pas envisageable d'explorer tous les coups possibles. C'est pour cette raison qu'il est primordial d'ajouter le **élagage Alpha-beta**. De plus, pour que la contrainte des 5 minutes soit respectée, il faut également imposer une profondeur maximum de l'arbre. Pour cela on va utiliser l'**Iterative Deepening Search** qui évite d'avoir une profondeur fixe. L'agent Minimax basique pour ce projet contient donc ces deux ajouts.

Pour améliorer ensuite l'agent, de multiples ajouts ont eu lieu :

- **Move ordering** : Triage des coups par score heuristique décroissant afin que alpha-beta voit d'abord les meilleurs coups ;
- **Killer moves** : Lorsque un coup provoque un cutoff beta, on mémorise la profondeur, et au prochain noeud à la même profondeur, ce coup sera testé en premier lieu ; (Chessprogramming Wiki, 2021)
- **Table de transposition (Zobrist)** : Puisque un même coup peut être atteint par plusieurs chemins différents, une table de transposition est utilisée afin d'éviter de calculer à nouveau les mêmes noeuds. Pour cela, le hash de Zobrist (Chessprogramming Wiki, 2026) est utilisé. Le hash d'un état est fait par un XOR sur tous les nombres correspondant aux pièces sur la grille. Le point intéressant est que si une pièce est ajoutée ou retirée, il suffit de XOR avec son nombre, et non de recalculer le hash en entier.
- **Quiescence search** : Cette recherche a lieu lorsque alpha-beta arrive en profondeur 0. Au lieu d'évaluer normalement et retourner le score, cette recherche va regarder si quelqu'un peut gagner immédiatement depuis l'état actuel du jeu, ou bien au prochain coup, puis on retourne le meilleur score. Le but est d'éviter l'effet horizon (lorsque l'agent a atteint sa profondeur maximale et qu'il ne peut pas voir la suite). La profondeur de la quiescence est fixée à deux, car chercher plus loin serait d'une part plus long, mais de plus quasiment inutile.
- **Singular extension** : Dans alpha-beta, les coups sont explorés à profondeur-1, mais il arrive qu'un coup est bien meilleur que les autres. L'explorer à la même profondeur que les autres coups serait du gaspillage. Si le score de ce coup moins le score du deuxième meilleur coup est supérieur à une "marge singulière" alors ce coup est singulier. (Chessprogramming Wiki, 2024)
- **Probcut** : Alpha-beta explore beaucoup de branches qui vont être coupées, le but de probcut est de prédire celles qui le seront afin qu'on ne doivent pas les explorer complètement. Si même lors de cette recherche peu profonde un score bien au-dessus de beta est retourné, alors il est quasi certain que la branche sera coupé, donc autant la couper directement.
- **Aspiration windows** : Lors de l'iterative deepening, au lieu de chercher dans un fenêtre allant de moins l'infini à plus l'infini, on part d'une fenêtre plus réduite autour du score de l'itération précédente. Si le score est stable entre les profondeurs, il y a moins de noeuds à explorer.

(Chessprogramming Wiki, 2022)

Une fonctionnalité d'amélioration n'a pas été retenue :

— **Late Move Reduction** : Les coups sont triés par score heuristique, logiquement ceux en fin de liste sont mauvais et ne valent pas la peine d'être explorés. Néanmoins, il faut faire attention où l'on place la limite entre le bon et le mauvais score.

2.2 Monte-Carlo Tree Search (MCTS)

MCTS construit l'arbre de jeu en répétant des simulations, allouant un nombre plus important aux branches intéressantes. Le MCTS est normalisé avec la valeur heuristique entre -1 et 1 et utilise des rollouts aléatoires pour évaluer. Il y a une fonctionnalité d'amélioration :

— **Priors heuristiques** : Au lieu d'avoir des priors uniformes, chaque coup est évalué et normalisé en probabilités.

Deux fonctionnalités d'améliorations non retenues :

— **Réutilisation de l'arbre** : Après un coup, au lieu de réinitialiser l'arbre, on avance la racine vers le noeud correspondant au coup joué et les simulations des tours précédents sont réutilisés ;

— **Truncated rollout** : On joue 5 coups aléatoires puis on évalue. L'idée est de donner une estimation plus dynamique en explorant un peu plus loin. Cette expérience était inspirée de la politique rollout d'AlphaGo (Silver et al., 2016).

2.3 Heuristique

Minimax et MCTS utilisent une heuristique pour évaluer l'état du jeu sans devoir aller jusqu'à la fin de la partie. Cette dernière retourne un score, positif si la position est favorable au joueur, négatif si elle est défavorable.

La base de l'heuristique suit les règles du jeu et contient :

— **Score des lignes** : Toutes les lignes (horizontales et verticales) possibles avec quatre cases consécutives sont parcourues. Pour chaque ligne, on regarde si cette dernière contient uniquement nos pièces, celles de l'adversaire, ou bien si c'est une ligne mixte. Dans le cas où il y a trois pièces alignées avec une quatrième case vide, c'est une menace immédiate de victoire et l'agent devra les prioriser.

— **Score des totems** : Contrôler le totem signifie qu'on peut le placer dans une zone stratégique du plateau avec des cases adjacentes vides utiles, et loin des zones où l'adversaire aimerait déposer le totem. Dès lors, le nombre de cases vides adjacentes au totems et leur qualité stratégique est évaluée.

En plus de cela, le comptage des pièces restantes a été implémenté mais non retenu. Le but de ce dernier était de reconnaitre un avantage pour un joueur si son adversaire avait moins de pièces "X" ou "O" de disponibles et qu'il ne pouvait donc pas profiter de certains alignements.

2.4 AlphaZero

En plus des agents guidés par des heuristiques faites à la main, nous avons voulu implémenter des agents basés sur des réseaux neuronaux. En premier lieu, nous avons choisi d'adapter l'implémentation d'AlphaZero (Silver et al., 2016, 2017) à Oxono.

Pour ce faire, nous avons implémenté le même réseau de neurones que celui utilisé par AlphaZero, à savoir un réseau de neurones convolutifs avec des blocs résiduels possédant deux têtes de sortie : une tête de politique pour prédire la probabilité de chaque action et une tête de valeur pour prédire l'évaluation de l'état.

De plus, il a fallu adapter la représentation de l'état du jeu pour qu'elle soit compatible avec le réseau de neurones. Nous avons utilisé un tenseur comportant 7 canaux, chacun de dimension 6×6, correspondant à la taille du plateau de jeu, où les canaux 1 et 2 encodent respectivement les positions des pions O de couleurs rose et noire, 3 et 4 sont dédiés aux X, 5 et 6 encodent la position des totems O et X, et le dernier canal encode le tour de jeu.

Ensuite, pour la sélection d'action, nous avons couplé le réseau de neurones à l'algorithme MCTS. La sortie de la tête de politique sert à guider la recherche, tandis que la sortie de la tête de valeur sert

à évaluer les nœuds feuilles de l'arbre. Durant les simulations, la sélection des nœuds s'effectue selon la formule PUCT (Predictor + Upper Confidence Bound applied to Trees) (équation 1) :

$$PUCT(s,a) = -Q(s,a) + c \cdot P(s,a)\frac{\sqrt{N(s)}}{1+N(s,a)} \tag{1}$$

Où $Q(s,a)$ est la valeur moyenne de l'action a évaluée depuis le point de vue du joueur suivant, $P(s,a)$ est la probabilité de l'action a donnée par la tête de politique, $N(s)$ est le nombre total de visites de l'état s, et $N(s,a)$ le nombre de visites de l'action a. L'hyperparamètre c contrôle l'exploration, il est fixé à 1.5 lors de la phase de jeu, mais augmenté à 2.5 durant l'entraînement pour garantir une meilleure diversité d'exploration.

Afin d'entraîner ce réseau de neurones sans aucune connaissance experte humaine, nous avons utilisé l'apprentissage par renforcement via auto-jeu, où le réseau joue continuellement contre lui-même pour générer ses propres données d'entraînement.

Pour finir, nous avons ajouté une table de Zobrist lors de la phase de jeu pour accélérer les évaluations des positions déjà rencontrées.

2.5 Athénan

En plus de l'implémentation d'AlphaZero, nous avons également implémenté un autre agent basé sur un réseau de neurones, mais qui, cette fois-ci, utilise Minimax pour la sélection d'action, Athénan (Cohen-Solal and Cazenave, 2025).

Pour commencer, le réseau d'Athénan n'a plus besoin de tête de politique, il ne possède donc qu'une tête de valeur pour évaluer les positions. De plus, un changement d'architecture a été fait pour opter pour un réseau R2, qui est détaillé dans le tableau 1.

Table 1 – Architecture du réseau de neurones *R2-network* implémenté.

Couche	**Type**	**Détails et paramètres**
1	Convolution	Filtres : 132, Noyau : 3×3, avec padding.
2 – 9	8 Blocs Résiduels	Par bloc : ReLU $\rightarrow$ Conv (3×3, 132 filtres) $\rightarrow$ ReLU $\rightarrow$ Conv (3×3, 132 filtres).
10	Dense + ReLU	518 neurones.
11	Dense + ReLU	518 neurones.
12	Dense (Sortie)	1 neurone (évaluation de la valeur de l'état).

Les différents paramètres de ce réseau ont été choisis afin de satisfaire un choix fait dans l'article d'Athénan, à savoir que le réseau doit contenir environ 5 millions de paramètres.

Ensuite, nous utilisons deux algorithmes de recherche différents pour la phase de jeu et la phase d'entraînement. Durant la phase de jeu, nous utilisons un Unbounded Best-First Minimax qui étend, de manière itérative, l'arbre de recherche en profondeur pour trouver la meilleure action à jouer, le tout utilisant une table de Zobrist pour accélérer les évaluations des positions déjà rencontrées. À la fin du temps imparti pour la recherche, l'action choisie est celle qui a le plus de visites. Durant la phase d'entraînement, nous utilisons Descent Minimax, qui effectue une recherche en profondeur de l'arbre de jeu jusqu'à atteindre un état terminal du jeu. Une fois cette recherche effectuée, l'algorithme utilise un processus appelé Tree Learning pour mettre à jour les poids du réseau de neurones. Contrairement à AlphaZero qui se limite au Terminal Learning (c'est-à-dire l'apprentissage exclusif sur les états constituant la séquence finale de la partie, avec pour cible le résultat final du match), Athénan apprend sur l'intégralité des états de l'arbre de recherche partiel généré par l'algorithme. De plus, la cible d'apprentissage n'est pas l'issue finale de la partie, mais la valeur Minimax partielle calculée pour chaque état de l'arbre.

Cette approche permet de n'avoir aucune perte d'information issue de la recherche et de générer environ 296 fois plus de données d'entraînement pour un même nombre de parties jouées, permettant ainsi un entraînement plus rapide et plus efficace du réseau de neurones.

3 Résultats et discussions

Pour l'ensemble de nos tests et le tournoi final, les agents disposent d'un budget strict de 5 minutes par partie. Afin de respecter cette contrainte, le temps de réflexion est alloué dynamiquement à chaque tour en divisant le temps restant par le nombre de pièces encore en possession du joueur. De plus, avant toute recherche profonde, l'agent vérifie systématiquement l'existence d'un coup gagnant immédiat pour économiser son temps.

3.1 Évolution de Minimax

Le tableau 2 valide notre étude d'ablation : la majorité des optimisations maintient un taux de victoire parfait (20-0). À l'inverse, la *Last Move Reduction* a engendré des erreurs d'évaluation fatales (4 défaites) et a donc été logiquement écartée pour préserver la fiabilité de l'agent.

Table 2 – Résultats détaillés de l'évolution de Minimax vs MCTS basique (20 parties par match).

Agent	**V**	**D**	**N**	**T/C**	**T/P**
Minimax basique	9	11	21,4	17,98	381,8
+ Move ordering	20	0	21	17,05	375,2
+ Killer moves	20	0	20,4	17	364
+ Table de transposition	20	0	19	16,88	337,8
+ Quiescence search	20	0	21	17,05	375,2
+ Last move reduction	16	4	18,7	17,21	332,2
+ Singular extension	20	0	21,5	17,05	385
+ Probcut	20	0	21,5	17,05	384,7
+ Aspiration windows	20	0	22,5	17,12	403,3

V : Victoire, D : Défaite, N : Nombre de coups moyen, T/C : Temps par coup moyen(sec.), T/P : Temps moyen par partie

3.2 Évolution de MCTS

Comme illustré au tableau 3, l'intégration des *priors heuristiques* a drastiquement amélioré les performances (passant de 9 à 15 victoires). En revanche, la réutilisation de l'arbre et les *early playouts* ont totalement dégradé le niveau de jeu (chute à 0 victoire), justifiant leur abandon immédiat.

Table 3 – Résultats détaillés de l'évolution de MCTS vs Minimax basique (20 parties par match).

Agent	**V**	**D**	**N**	**T/C**	**T/P**
MCTS basique	9	11	19,6	18,02	350,8
+ Priors heuristique	15	5	14,1	16,89	245
+ Réutilisation arbre	8	12	12,9	17,66	223,4
+ Early playout	0	20	18,1	18,78	320,9

3.3 Évolution de l'heuristique

Le tableau 4 démontre l'importance de la dimension spatiale dans Oxono : évaluer le contrôle des totems fait bondir le taux de victoire à 14. À l'inverse, le simple comptage des pièces restantes s'est avéré totalement contre-productif face à un adversaire optimisant le plateau (0 victoire) et a été retiré.

Table 4 – Résultats détaillés de l'évolution de l'heuristique avec MCTS avancé vs Minimax basique (20 parties par match).

Agent	**V**	**D**	**N**	**T/C**	**T/P**
Heuristique basique	3	17	12,6	18,36	216,9
+ Pièces restantes	0	20	15,9	18,79	279,7
+ Contrôle totems	14	6	14,8	17,1	259,1

3.4 Tournoi suisse

Afin d'évaluer les performances de nos différents agents, nous avons organisé un tournoi suisse à 4 rondes. Nous avons inclus dans ce tournoi les agents suivants : Minimax, MCTS, AlphaZero, Athénan, ainsi que le Random agent. A chaque ronde, les agents s'affrontent en 1 contre 1 sur 20 parties (10 parties de chaque côté). Une victoire apporte 3 points, une défaite 0 point, et un match nul 1 point à chaque agent. Etant donné que nous avons 5 agents, un agent ne joue pas à chaque ronde, et reçoit donc un point de bye (ici correspondant à avoir fait un match nul à chaque partie, soit 20 points). Les résultats de ce tournoi sont présentés dans le tableau 5.

Table 5 – Résultats détaillés du tournoi suisse (4 rondes, 20 parties par match).

Rang	**Agent**	**R1**	**R2**	**R3**	**R4**	**Total**
1	Minimax	60	36	30	60	**186**
2	Athénan	39	24	20*	57	**140**
3	MCTS	21	60	30	20*	**131**
4	AlphaZero	0	20*	51	3	**74**
5	Random	20*	0	9	0	**29**

* Indique un point de *bye* (20 points accordés d'office).

Pour information, l'agent Athénan et AlphaZero ont été entraînés pendant environ 48 heures chacun. Athénan a été entraîné sur CPU (AMD Ryzen 7 5700X avec 64Go de RAM disponible), tandis qu'AlphaZero a été entraîné sur GPU (NVIDIA GeForce RTX 4060 Ti avec 8Go de VRAM disponible).

Avec les résultats du tournoi, nous pouvons observer que les approches Minimax, incluant Minimax et Athénan, sont plus performantes que les approches basées sur MCTS, à savoir MCTS et AlphaZero.

De plus, il est intéressant de noter que les agents basés sur des réseaux de neurones, à savoir AlphaZero et Athénan, ont des performances inférieures à celles de leurs homologues basés sur des heuristiques faites à la main, Minimax et MCTS respectivement. Cependant, il est important de souligner que les agents basés sur des réseaux de neurones ont été entraînés pendant une période relativement courte (environ 48 heures), et qu'avec plus de temps d'entraînement, leurs performances pourraient potentiellement s'améliorer. De plus l'entraînement d'AlphaZero a été réalisé avec un équipement beaucoup plus limité que celui utilisé réellement par DeepMind pour entraîner leur agent, ce qui a pu limiter les performances de notre implémentation d'AlphaZero.

4 Conclusion

Au cours de ce projet, nous avons implémenté et évalué plusieurs agents pour le jeu Oxono, en utilisant différentes approches telles que Minimax, MCTS, et des réseaux de neurones inspirés d'AlphaZero et d'Athénan. Les résultats ont montré que les agents basés sur des heuristiques faites à la main ont surpassé ceux basés sur des réseaux de neurones, bien que ces derniers aient été entraînés pendant une période relativement courte. Notre étude a également mis en comparaison différentes heuristiques et améliorations, démontrant l'importance de leur influence sur les différents agents.

Basés sur ces résultats, nous avons donc décidé d'utiliser l'agent Minimax pour le premier rendu, car il garantit les meilleures performances immédiates. Cependant, conscients du fort potentiel de l'apprentissage par *Tree Learning* d'Athénan, nous sommes en parallèle en train de continuer son entraînement avec pour ambition de l'utiliser pour la compétition finale.

Références

Chessprogramming Wiki. Killer heuristic, 2021. URL https://www.chessprogramming.org/Killer_Heuristic.

Chessprogramming Wiki. Aspiration windows, 2022. URL https://www.chessprogramming.org/Aspiration_Windows.

Chessprogramming Wiki. Singular extensions, 2024. URL https://www.chessprogramming.org/Singular_Extensions.

Chessprogramming Wiki. Zobrist hashing, 2026. URL https://www.chessprogramming.org/Zobrist_Hashing.

Quentin Cohen-Solal and Tristan Cazenave. Minimax strikes back, 2025. URL https://arxiv.org/abs/2012.10700.

Stuart Russell and Peter Norvig. *Artificial Intelligence : A Modern Approach*. Pearson Series in Artificial Intelligence. Pearson, 4 edition, 2021. ISBN 978-1-292-40113-3.

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587) :484–489, 2016. doi : 10.1038/nature16961. URL https://doi.org/10.1038/nature16961.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm, 2017. URL https://arxiv.org/abs/1712.01815.